SOFTWARE ARCHITECTURE — CHAPTER 10

Scalability

Growing workloads, bounded resources — and architecture that keeps latency under control

Based on Pautasso — Visual Lecture Notes

Full **What / Why** presenter bullets: `Lecture10.pptx` or `LECTURE_SLIDES_marp.html` (from Marp).

Chapter PDF: `10_Scalability.pdf`

RECAP — LECTURE 9

Deployability & Containers

Repeatable images, pipelines, portability — we can ship often and run the same artifact everywhere

→ Today: once deploys are **easy**, **demand** becomes the next limiter: throughput, data, and cost.

Chapter roadmap (Pautasso)

- **Workloads and resources** — what grows vs what you buy
- **Scale up vs scale out** — vertical limits; horizontal economics
- **Scaling dimensions** — clients, payload size, state, dependencies
- **Location transparency** — discovery, load balancing, injection
- **Patterns** — scatter/gather, master/worker, sharding, caching

Learning objectives

- Define **scalability** using workload and resource language
- Contrast **scale up** vs **scale out** for typical tiers (API, DB, workers)
- Spot **serial bottlenecks** — locks, single writer, hot keys, fan-out
- Relate **caches**, **queues**, and **partitions** to CityBite peak traffic
- Name a few **runtime patterns** and when they help or hurt operations

What is scalability?

Scalability = ability to handle **growth** in workload (users, data, events) by adding **resources** while meeting targets for latency, throughput, and cost.

- Not “always faster” — often “same p95 latency at 10× orders/min”
- Architecture decides whether extra machines give **linear** capacity or waste money

CityBite: Friday dinner + coupon push = concurrent checkouts + kitchen refreshes + dispatch map queries.

Workload vs resources

Workload (demand)

Requests/sec, bytes in/out, DB rows touched, background jobs, cross-region replication…

Resources (supply)

vCPU, RAM, disk IOPS, network egress, connection pools, thread pools, people on-call.

Saturation of **any one** resource becomes the system bottleneck.

Throughput vs latency (Pautasso)

- **Linear region:** response time stays flat; throughput rises with workload
- **Saturation:** throughput **plateaus**; queues grow → latency climbs
- **Overload:** timeouts, retries, **retry storms**, drops — unstable

Do not confuse throughput with latency. **Infinite scalability does not exist** — only a useful range before saturation.

Scale levers: faster, less, later, help

- **Work faster** — better algorithms, bigger CPUs (until diminishing returns)
- **Work less** — approximations; **degrade** non-critical features during peak
- **Work later** — **queues**; buffer the spike; drain when idle
- **Get help** — scale out; parallelism; more workers

Also: **cache** (avoid recomputation), **edge offload** (maps, previews on device) — same ideas as Netflix/YouTube edge.

Scale up vs scale out

	Scale up	Scale out
Idea	Bigger machine	More machines / pods
Pros	Simpler topology; strong single-node DB	Fault isolation; elastic with HPA
Cons	Ceiling; expensive; blast radius	Needs **stateless** or careful partitioning

Weak vs strong scaling

- **Strong scaling** — problem size fixed; more resources ⇒ shorter time per job
- **Weak scaling** — problem grows with resources; each node keeps similar local work

Bad decomposition: doubling pods does **not** halve latency if every pod hits the same lock or hot row.

Centralized vs decentralized

	Centralized	Decentralized
Consistency	Easier — one logical truth	Replication / consensus cost
Failure	**SPoF** — all-or-nothing	**Partial** failure; harder recovery
Other	Obvious bottleneck	**Hot spots**, **churn** in P2P

You can still **replicate** servers behind a logical service — “many boxes,” one product (e.g. Google front ends).

Famous systems (same patterns)

- **Netflix / YouTube** — edge CDN **cache**; encoding = **master/worker** on video chunks
- **Google Search** — **scatter/gather** over **index shards**; ranking = gather bottleneck
- **Amazon** — service boundaries; **Dynamo**-style partitioning for massive key-value scale
- **Uber / Lyft** — **geo**-oriented dispatch; surge pricing = capacity + economics
- **X (Twitter)** — **hot accounts** = shard/cache hotspot; timeline **fan-out**

Scalability at scale — graph limits

- Each node has finite **connections** and **bandwidth** — “black hub” or full mesh, both hit **N**
- **Hierarchy (tree)** — root becomes the next bottleneck (fan-in)
- **Super-peers / overlays** — stable hubs + neighborhoods (classic P2P lesson)
- **Gossip** — resilient under churn; harder to visualize and debug

Removing the center is not enough if every node still behaves like the old center.

Scaling dimensions

- **Clients** — concurrent sessions, mobile retries, WebSocket dashboards
- **Input size** — search query breadth, menu JSON, map tiles
- **State** — carts, active orders, driver positions, idempotency keys
- **Dependencies** — payment gateway rate limits, maps API quotas, partner webhooks

CityBite: dinner rush story

- **Customer spike** — checkout, ETA, tracking poll every few seconds
- **Restaurant spike** — kitchen board refresh; printers; “mark ready” bursts
- **Dispatch** — map + assignment reads; fewer writes but heavy reads
- Risk: one subsystem (often **OLTP primary**) caps whole experience

Bottleneck archetypes

- **CPU** — serialization, JSON, crypto, image transforms on hot path
- **I/O** — disk logs, blocking DB calls, synchronous partner APIs
- **Locks** — in-process mutex; DB row contention on popular restaurant
- **Pool exhaustion** — HTTP threads, DB connections, outbound sockets

Stateless tiers scale “easily”

If a pod holds **no** unique session memory, the load balancer can spray requests.

- Externalize session to Redis or signed cookie + server-side idempotency
- Uploads and large caches belong on **object storage** / CDN — not pod disk

Lecture 9 + 10 together: disposable pods **and** data planes that survive churn.

When you need affinity

- **Sticky sessions** — sometimes legacy; prefer external session store
- **Partition-aware routing** — shard key → subset of nodes (advanced)
- Trade-off: affinity can create **hot shards** if one tenant goes viral

Load balancing

- Distribute work across healthy replicas — L4/L7, health-aware
- Algorithms: round-robin, least connections, weighted, consistent hashing (for caches)
- LB hides individual pod failure — if app is **slow**, LB amplifies retry storms unless tuned
- **Production** examples: AWS ALB/NLB, GCP GLB, Cloudflare in front of GitHub, Shopify, global APIs

Location transparency — directory vs DI

- **Directory / registry** — client (active) looks up provider location; registry passive
- **Dependency injection** — container (active) wires dependencies; business code passive
- Same problem as **“which shard holds this key?”** — indirection + metadata

Shard routing tables, service mesh, ZooKeeper — all flavors of **discovery**.

Database: the usual serial line

- Single **primary** for OLTP writes — fundamental serialization of truth
- **Read replicas** — scale read-heavy dashboards; replication lag = staleness budget
- Connection pooling per pod × replica count = watch **max_connections**

Caching

- **Menu / static** — CDN or edge cache; version in URL or ETag
- **Read-through** Redis for “hot” restaurant configs — TTL + explicit invalidation on update
- Stale cache can be OK for ETAs; **not** OK for payment or inventory if you promised ACID
- **Spotify** metadata, **Netflix** Open Connect — same pattern: immutable-ish catalogs + TTL discipline

Queues and workers

- **Absorb spikes** — accept writes fast; drain async work at sustainable rate
- Examples: notifications, search index updates, fraud scoring, settlement batches
- Need **backpressure** — max queue depth, drop vs delay, DLQ for poison messages

See `example2_scalability_queue_workers_citybite.py`

Partitioning / sharding

- Split data by **key** — e.g. `restaurant_id`, city, tenant
- Queries must carry the key — cross-shard joins are expensive
- Rebalancing shards is an **architecture migration** — plan early boundaries

See `example1_scalability_hot_path_citybite.py` — same idea as DB indexes / partition pruning.

Shard key strategies

- **Key ranges** — e.g. user id 0–500k / 500k–1M on two shards
- **Geo** — country/region (watch skew: populous vs tiny regions)
- **Time** — monthly logs; “this quarter” queries fan to many shards
- **Hash / modulo** — spread keys; changing shard count ⇒ **expensive resharding**

Often need a **directory** (logical shard id → physical host): ZooKeeper, etcd, managed metadata.

Sharding & blockchains

Blockchain sharding (e.g. **Ethereum** scaling roadmap, **Polkadot** parachains, **Cosmos** zones) partitions **execution or state** so not every validator repeats **every** transaction.

- **Same idea** as DB sharding: **shard key**, routing, avoid “query hits all shards”
- **Extra difficulty** — Byzantine peers, consensus, **cross-shard atomicity** (bridges, 2PC-like protocols, fraud proofs)
- **Contrast** — **MongoDB** / **Citus** shard mainly for **IOPS + storage** under trusted operators; chains add **trust + game theory** cost

Industry names for partitions

Term	Example
Shard	MongoDB, Elasticsearch, SolrCloud
Region	HBase
Tablet	Bigtable / Colossus stack
vnode	Cassandra, Riak

Discord (public engineering posts) is a well-documented example of aggressive data-layer scaling at social scale.

Scatter / gather

- **Scatter** — fan query to many workers or shards (e.g. “search dishes citywide”)
- **Gather** — merge, rank, dedupe — often the new bottleneck
- CityBite: marketing “trending near you” could fan out to regional indices
- **Kayak / Google Flights** — scatter price requests to airlines; **gather** under a deadline (lecture’s “≤ 2 minutes” rule)

Scatter/gather — when to respond?

- Wait for **all** — latency = **slowest** replica
- **Deadline** — return best offer collected when time window closes
- **First k of n** — enough coverage without waiting for stragglers
- **Fastest "good enough"** — validate quality (avoid super-fast garbage answers)
- **Majority vote** — useful under **Byzantine** or flaky backends

Master / worker

- Coordinator assigns units of work — dispatch batching, nightly settlement
- Workers must be **idempotent** — retries happen under failures and autoscaling
- Master availability becomes critical — often multiple masters with consensus (out of scope detail)
- **Netflix** transcode, **movie render farms**, **AWS Batch** — same divide → merge shape

Master/worker — trust at internet scale

SETI@home (lecture): volunteer workers worldwide; popularity created **cheating** clients returning junk for leaderboard points.

- Mitigation: assign the **same partition** to multiple workers and **compare** (scatter/gather for **integrity**)
- Expectation on **minimum plausible** compute time — “too fast” is suspicious

Same lesson as Byzantine-tolerant aggregation in scatter/gather — **trust** is part of scale design.

Anti-pattern: shared mutable server memory

- Shopping cart only in one pod's RAM → breaks horizontal scale or needs sticky routing
- In-memory "global order board" for all restaurants → every query scans the world

Prefer explicit stores with clear **consistency** and **partition** story.

Autoscaling (Kubernetes tie-in)

- **HPA** on CPU, memory, or custom metrics (e.g. requests in flight)
- Scaling **pods** without scaling **Postgres** → thinner app, same DB wall
- Combine with SLOs — e.g. target p95 checkout < 2s at peak coupon traffic

Measure before you myth-bust

- **Percentiles** — p95/p99 latency; mean hides tail disasters
- **Saturation** — CPU, pool wait time, queue depth, replication lag
- **Load tests** — scripted dinner rush; compare to production guardrails

References

- Martin L. Abbott & Michael T. Fisher — *The Art of Scalability* (Pearson)
- Gregor Hohpe & Bobby Woolf — *Enterprise Integration Patterns*
- Martin Kleppmann — *Designing Data-Intensive Applications* (O'Reilly)

Chapter notes transcript: `lectrue10.txt` (aligns with Pautasso PDF).

Example 1 — hot path (CityBite kitchen)

`example1_scalability_hot_path_citybite.py`

- Naive: scan **all** orders for every restaurant query
- Better: **index** by `restaurant_id` — same lesson as DB partition keys

Run: `python3 example1_scalability_hot_path_citybite.py`

Example 2 — checkout vs notify (CityBite)

`example2_scalability_queue_workers_citybite.py`

- Anti-pattern: block checkout on every SMS/email
- Better: **enqueue** + parallel workers — decouple user latency from outbound I/O

Run: `python3 example2_scalability_queue_workers_citybite.py`

Assignment — CityBite at peak

150 points — document how CityBite scales under growth (workload model, scale decisions, data plane, diagrams, patterns, HPA/backpressure).

- Parts 1–3: markdown + `.drawio` + `.png`
- Reuse **CityBite** naming and Lecture 9 baseline (K8s, Postgres, uploads)

Full rubric: `ASSIGNMENT.md`

Before you submit

1. Run both Python examples; quote **one number** from each in your Part 1 or Part 2 narrative
2. Mark every “10× traffic” claim with **what breaks first** (DB, pool, queue, …)
3. Diagram must differ **steady vs peak** — not only more arrows

Takeaways

- Scalability = **workload dimensions** vs **resources**; find the serial bottleneck
- Scale out needs **partitioned work** or **replicated read-mostly** paths
- **Caches** and **queues** buy time and decouple peaks — with explicit consistency and depth limits
- **Shards** and **scatter/gather** trade query freedom for raw size
- Observe **p95** and **saturation**; autoscale the tier that actually limits SLOs
- **Shard keys** + directories — same routing problem as **blockchain** shards, simpler trust in owned DBs

Questions?

Chapter 10 — Scalability

Deep dive: `10_Scalability.pdf` · Notes: `lectrue10.txt`

Next arc (course): availability, flexibility, and failure design.